

CSC311: Data Structures

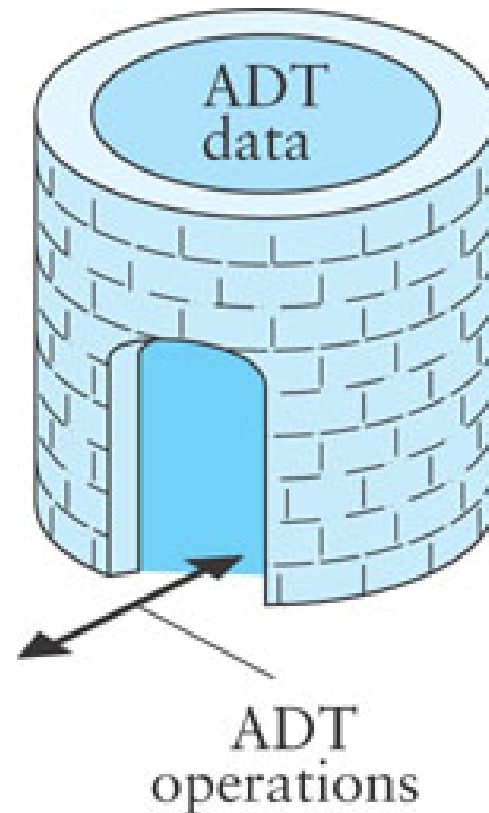
ADTs, Arrays, ArrayList

Dr. Amlan Chatterjee

Computer Science Department
California State University, Dominguez Hills

ADTs

- Abstract Data Type (ADT)
- An encapsulation of data and methods
- Allows for reusable code
- The user need not know about the implementation of the ADT
- A user interacts with the ADT using only public methods



ADTs (cont.)

- ADTs facilitate storage, organization, and processing of information
- Such ADTs often are called *data structures*
- The Java Collections Framework provides implementations of common data structures

Interfaces

- An interface specifies or describes an ADT to the applications programmer:
 - the methods and the actions that they must perform
 - what arguments, if any, must be passed to each method
 - what result the method will return
- The interface can be viewed as a *contract* which guarantees how the ADT will function

Interfaces (cont.)

- A class that *implements the interface* provides code for the ADT
- As long as the implementation satisfies the ADT contract, the programmer may implement it as he or she chooses
- In addition to implementing all data fields and methods in the interface, the programmer may add:
 - data fields not in the implementation
 - methods not in the implementation
 - constructors (an interface cannot contain constructors because it cannot be instantiated)

Interfaces (cont.)

- The interface definition shows only headings for its methods
- Because only headings are shown, they are considered *abstract methods*
- Each abstract method must be defined in a class that implements the interface

The implements Clause

- For a class to implement an interface, it must end with the `implements` clause

```
public class ATMBankAmerica implements ATM  
public class ATMforAllBanks implements ATM
```

- A class may implement more than one interface—their names are separated by commas

Array Data Structure

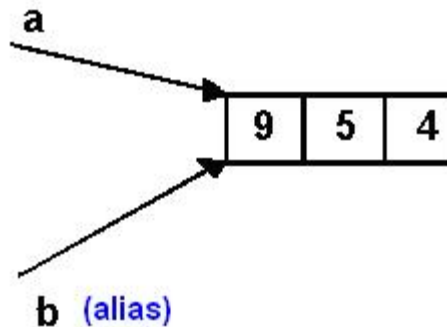
- An array is an aggregate data structure that is designed to store a group of objects of the same type.
- Arrays can hold primitives as well as references.
- The array is the most efficient data structure for storing and accessing a sequence of objects.
- Here is the list of most important array features you must know (i.e., be able to program)
 - copying
 - insertion and deletion
 - searching and sorting

Array Data Structure

- You already know that the Java language has only two data types, primitives and references.
- Which one is an array? Is it primitive?
 - An array is not a primitive data type - it has a field (and only one), called *length*.
- Therefore, you deal with arrays as you deal with references.
- One of the major differences between references and primitives is that you cannot copy arrays by assigning one to another:
 - `int[ ] a = {9, 5, 4};`
 - `int[ ] b = a;`

Array Data Structures

- The assignment operator creates an alias to the object, like in the picture below



- Since these two references **a** and **b** refer to the same object, comparing them with the double equal sign "==" will always return true.
- In the next code example, `int [] a = {1,2,3}; int [] b = {1,2,3};` **a** and **b** refer to two different objects (though with identical contents).
- Comparing them with the double equal sign will return false. How would you compare two objects with identical contents?
- In short, using the equals method. For array comparison, the Java APIs provides the *Arrays* class.

The Arrays Class

- The *java.util.Arrays* class is a convenience class for various array manipulations, like comparison, searching, printing, sorting and others.
- Basically, this class is a set of *static* methods that are all useful for working with arrays.
- The code below demonstrates a proper invocation of equals:

```
int[ ] a = {1,2,3};  
int[ ] b = {1,2,3};  
if( Arrays.equals(a, b) )  
    System.out.println("arrays with identical contents");
```

The Arrays Class (cont.)

- Another commonly used method is `toString()` which takes care of printing

```
int[ ] a = {1,2,3};  
System.out.println(Arrays.toString(a));
```

- Here is the example of sorting

```
int[ ] a = {3,2,1};  
Arrays.sort(a);  
System.out.println(Arrays.toString(a));
```

- In addition to that, the class has other utility methods for supporting operations over multidimensional arrays.

Copying Arrays

- There are different ways to copy arrays
 - using a loop structure
 - using `System.arraycopy()`
 - etc.

- The first way is very well known to you

```
int[ ] a = {1, 2, 3};  
int[ ] b = new int[a.length];  
for(int i = 0; i < a.length; i++)  
    b[i] = a[i];
```

Copying Arrays

- The most efficient copying data between arrays is provided by `System.arraycopy()` method.
 - The method requires five arguments.
 - `public static void arraycopy(Object source, int srcIndex, Object destination, int destIndex, int length)`
- The method copies `length` elements from a source array starting with the index `srcIndex` to a new array destination at the index `destIndex`.
- The above code example can be rewritten as follows:

```
int[ ] a = {1, 2, 3};  
int[ ] b = new int[a.length];  
System.arraycopy(a, 0, b, 0, 3);
```

Arrays: Insertion and Deletion

- Arrays in Java have no methods and only one *immutable* field length.
- Once an array is created, its length is fixed and cannot be changed.
- What do you do to resize the array?
 - You allocate the array with a different size and copy the contents of the old array to the new array.

Deletion from a primitive array

```
public char[ ] delete(char[ ] data, int pos)
{
    if(pos >= 0 && pos < data.length)
    {
        char[ ] tmp = new char[data.length-1];
        System.arraycopy(data, 0, tmp, 0, pos);
        System.arraycopy(data, pos+1, tmp, pos, data.length-pos-1);
        return tmp;
    }
    else
        return data;
}
```

The first arraycopy copies the elements from index 0 to index pos-1, the second arraycopy copies the elements from index pos+1 to data.length.

Interview Questions:

- How do you sort an array containing the ages of the students at a University?
 - What is the time complexity of your code?

The ArrayList class

- The *java.util.ArrayList* class supports an idea of a *dynamic* array - an array that grows and shrinks on demand to accommodate the number of elements in the array.
- Below is a list of commonly used methods
 - *add(object)* - adds to the end
 - *add(index, object)* - inserts at the index
 - *set(index, object)* - replaces at the index
 - *get(index)* - returns the element at that index
 - *remove(index)* - deletes the element at that index
 - *size()* - returns the number of elements

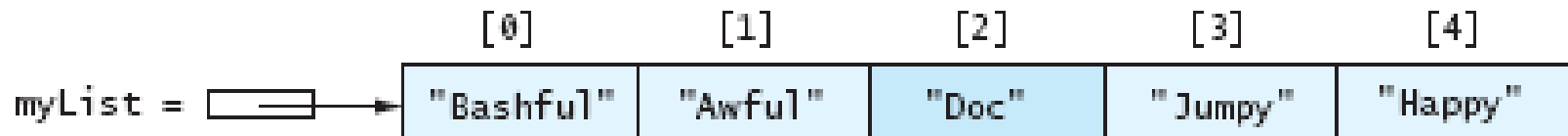
ArrayList Class (cont.)

- To declare a `List` “object” whose elements will reference `String` objects:
 - `List<String> myList = new ArrayList<String>();`
- The initial `List` is empty and has a default initial capacity of 10 elements
- To add strings to the list,
 - `myList.add("Bashful");`
 - `myList.add("Awful");`
 - `myList.add("Jumpy");`
 - `myList.add("Happy");`

ArrayList Class (cont.)



- Adding an element with subscript 2:
 - `myList.add(2, "Doc");`



After insertion of "Doc" before the third element

- Notice that the subscripts of "Jumpy" and "Happy" have changed from [2],[3] to [3],[4]

ArrayList Class (cont.)

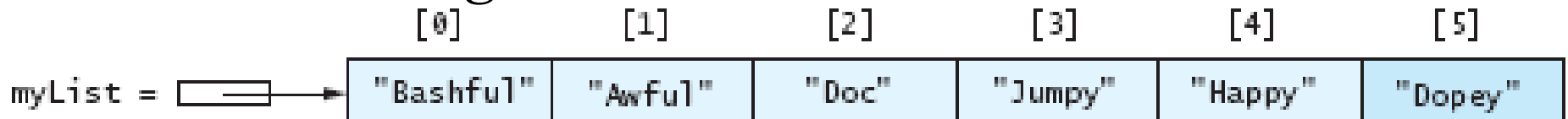
- When no subscript is specified, an element is added at the end of the list:

```
myList.add("Dopey");
```



ArrayList Class (cont.)

- Removing an element:



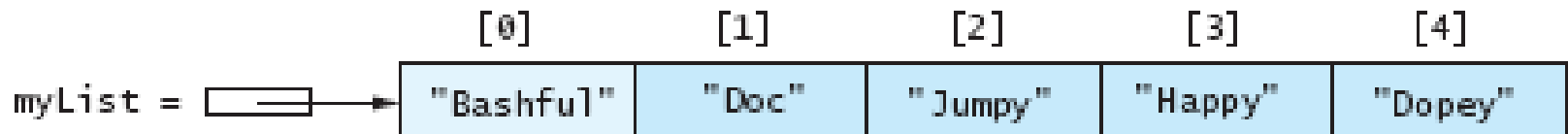
- `myList.remove(1);`



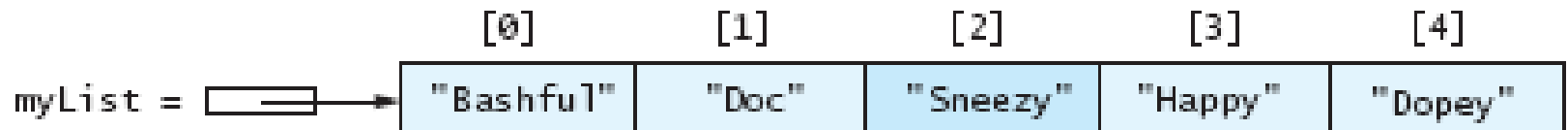
- The strings referenced by [2] to [5] have changed to [1] to [4]

ArrayList Class (cont.)

- You may also replace an element:

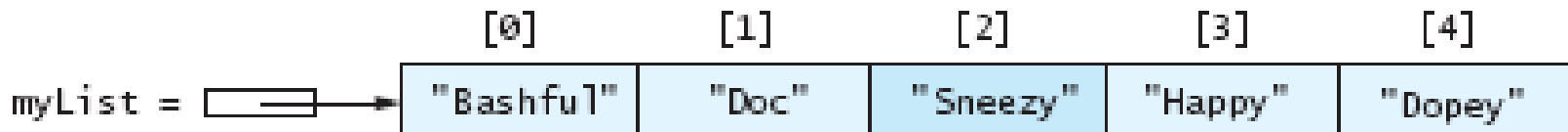


```
myList.set(2, "Sneezy");
```



After replacing "Jumpy" with "Sneezy"

ArrayList Class (cont.)

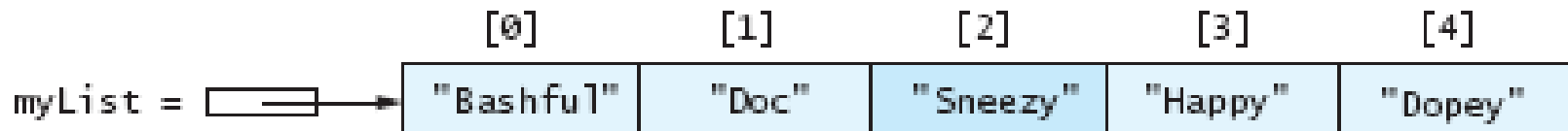


- You cannot access an element using a bracket index as you can with arrays (array[1])
- Instead, you must use the `get()` method:

```
String dwarf = myList.get(2);
```

- The value of `dwarf` becomes "Sneezy"

ArrayList Class (cont.)



- You can also search an ArrayList:
 - `myList.indexOf("Sneezy");`
- This returns 2 while
 - `myList.indexOf("Jumpy");`
- returns -1 which indicates an unsuccessful search

Generic Collections

- The statement

```
List<String> myList = new  
    ArrayList<String>();
```

uses a language feature called *generic collections* or *generics*

- The statement creates a List of String; only references of type String can be stored in the list
- String in this statement is called a *type parameter*
- The type parameter sets the data type of all objects stored in a collection

Generic Collections (cont.)

- The general declaration for generic collection is
 - *CollectionClassName*<E> variable =
new *CollectionClassName*<E>();
- The <E> indicates a type parameter
- Adding a noncompatible type to a generic collection will generate an error during compile time
- However, primitive types will be autoboxed:
 - `ArrayList<Integer> myList = new ArrayList<Integer>();`
 - `myList.add(new Integer(3)); // ok`
 - `myList.add(3); // also ok! 3 is automatically wrapped`
 - in an Integer object
 - `myList.add(new String("Hello")); // generates a type incompatibility error`

Why Use Generic Collections?

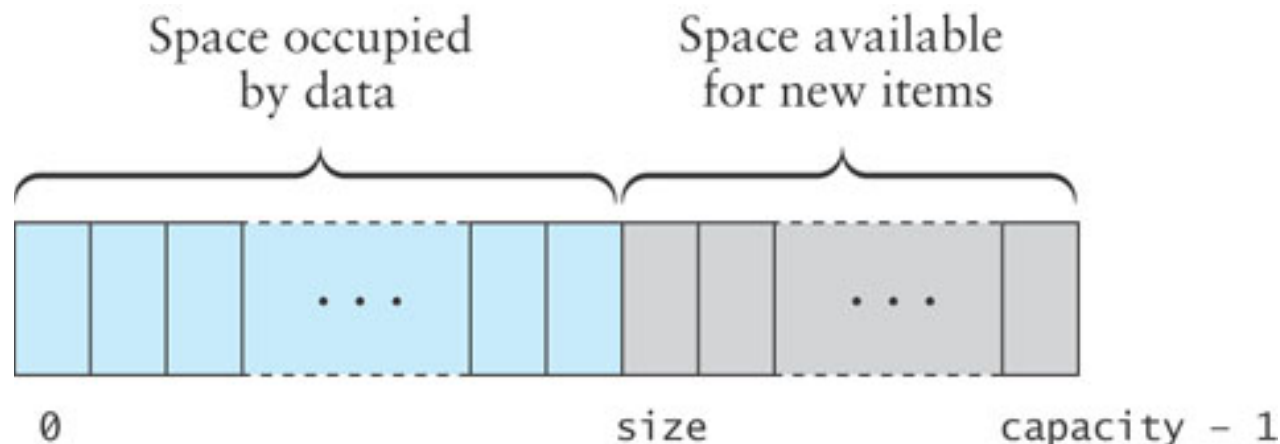
- Better type-checking: catch more errors, catch them earlier
- Documents intent
- Avoids the need to downcast from `Object`

Specification of the ArrayList Class

Method	Behavior
<code>public E get(int index)</code>	Returns a reference to the element at position <code>index</code> .
<code>public E set(int index, E anEntry)</code>	Sets the element at position <code>index</code> to reference <code>anEntry</code> . Returns the previous value.
<code>public int size()</code>	Gets the current size of the ArrayList.
<code>public boolean add(E anEntry)</code>	Adds a reference to <code>anEntry</code> at the end of the ArrayList. Always returns <code>true</code> .
<code>public void add(int index, E anEntry)</code>	Adds a reference to <code>anEntry</code> , inserting it before the item at position <code>index</code> .
<code>int indexOf(E target)</code>	Searches for <code>target</code> and returns the position of the first occurrence, or <code>-1</code> if it is not in the ArrayList.
<code>public E remove(int index)</code>	Returns and removes the item at position <code>index</code> and shifts the items that follow it to fill the vacated space.

Implementing an ArrayList Class

- DHArrayList: a simple implementation of ArrayList
 - Physical size of array indicated by data field *capacity*
 - Number of data items indicated by the data field *size*



DHArrayList Fields

```
import java.util.*;

/** This class implements some of the methods of the Java ArrayList
class
*/
public class DHArrayList<E> {
    // Data fields
    /** The default initial capacity */
    private static final int INITIAL_CAPACITY = 10;

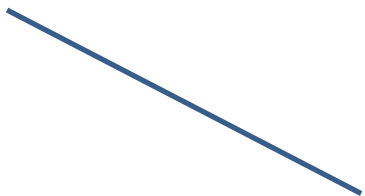
    /** The underlying data array */
    private E[] theData;

    /** The current size */
    private int size = 0;

    /** The current capacity */
    private int capacity = 0;
}
```

DHArrayList Constructor

```
public DHArrayList () {  
    capacity = INITIAL_CAPACITY;  
    theData = (E[]) new Object[capacity];  
}
```



This statement allocates storage for an array of type `Object` and then casts the array object to type `E[]`

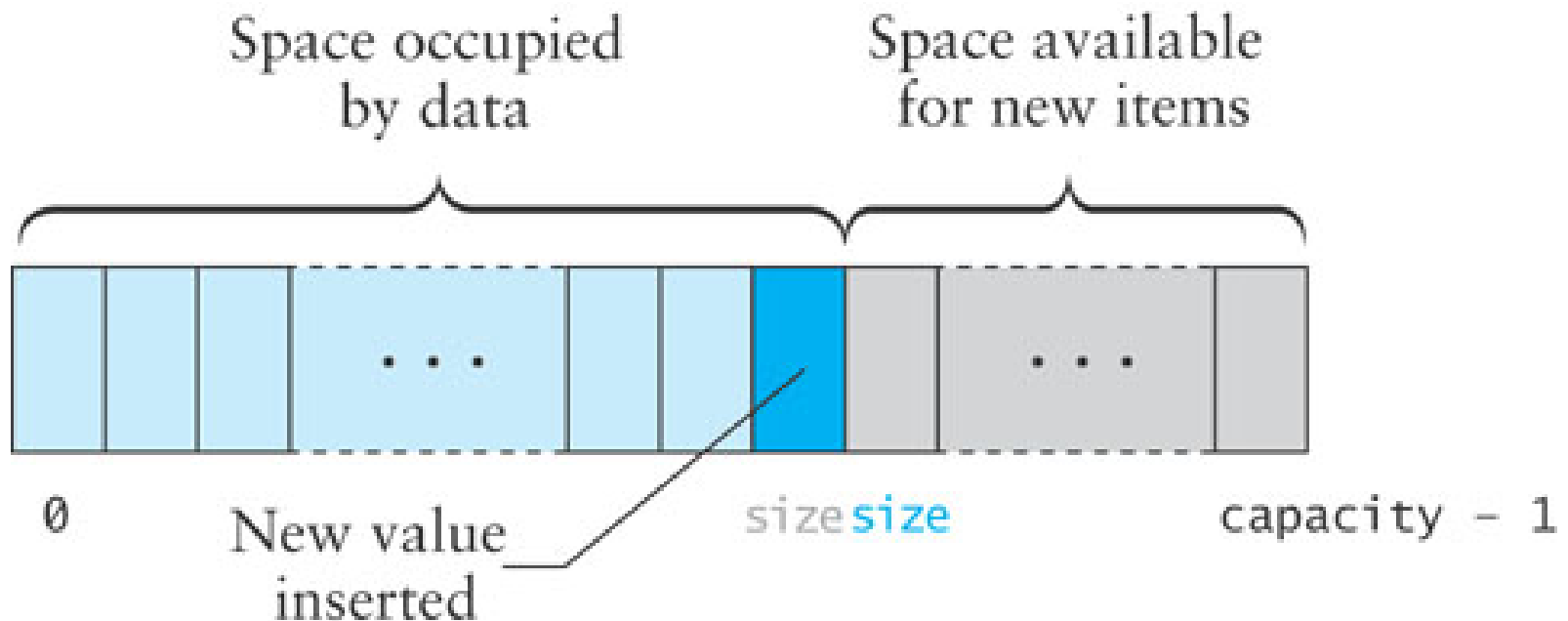
Although this may cause a compiler warning, it's ok

Implementing `ArrayList.add(E)`

- We will implement two add methods
- One will append at the end of the list
- The other will insert an item at a specified position

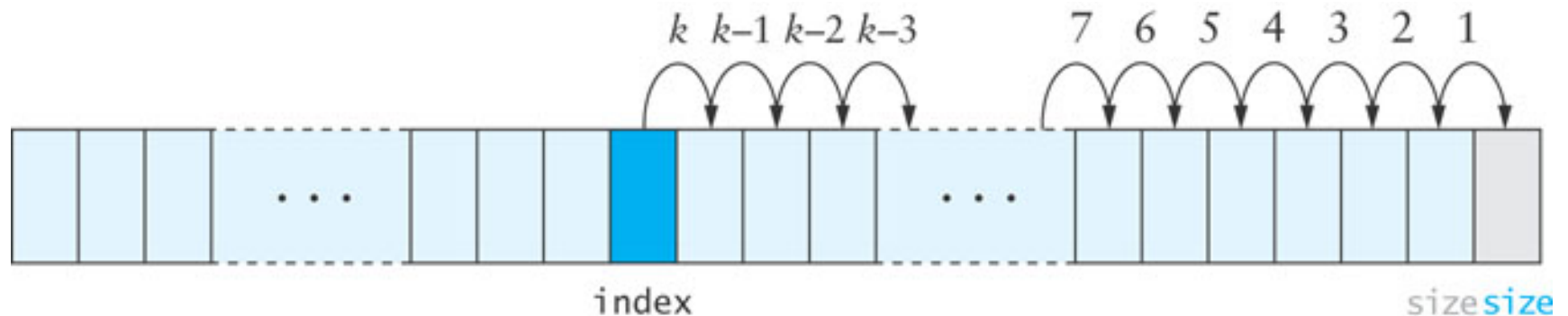
Implementing `ArrayList.add(E)`(cont.)

- If `size` is less than `capacity`, then to append a new item
 1. insert the new item at the position indicated by the value of `size`
 2. increment the value of `size`
 3. return `true` to indicate successful insertion



Implementing `ArrayList.add(int index, E anEntry)`

- To insert into the middle of the array, the values at the insertion point are shifted over to make room, beginning at the end of the array and proceeding in the indicated order



Implementing ArrayList.add(index, E)

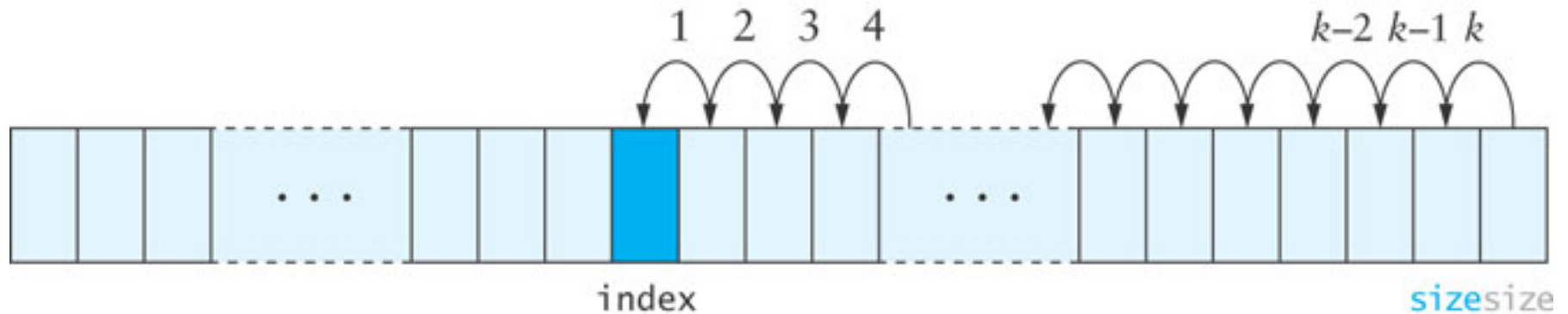
```
public void add (int index, E anEntry) {  
    // check bounds  
    if (index < 0 || index > size) {  
        throw new ArrayIndexOutOfBoundsException(index);  
    }  
  
    // Make sure there is room  
    if (size >= capacity) {  
        reallocate();  
    }  
  
    // shift data  
    for (int i = size; i > index; i--) {  
        theData[i] = theData[i-1];  
    }  
  
    // insert item  
    theData[index] = anEntry;  
    size++;  
}
```

set and get Methods

```
public E get (int index) {  
    if (index < 0 || index >= size) {  
        throw new ArrayIndexOutOfBoundsException(index);  
    }  
    return theData[index];  
}
```

```
public E set (int index, E newValue) {  
    if (index < 0 || index >= size) {  
        throw new ArrayIndexOutOfBoundsException(index);  
    }  
    E oldValue = theData[index];  
    theData[index] = newValue;  
    return oldValue;  
}
```

remove Method



- When an item is removed, the items that follow it must be moved forward to close the gap
- Begin with the item closest to the removed element and proceed in the indicated order

remove Method (cont.)

```
public E remove (int index) {  
    if (index < 0 || index >= size) {  
        throw new ArrayIndexOutOfBoundsException(index);  
    }  
  
    E returnValue = theData[index];  
  
    for (int i = index + 1; i < size; i++) {  
        theData[i-1] = theData[i];  
    }  
  
    size--;  
    return returnValue;  
}
```

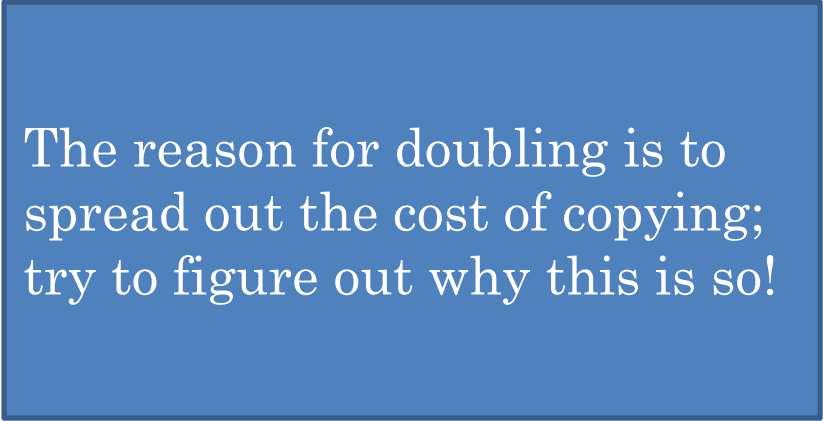
reallocate Method

- Create a new array that is twice the size of the current array and then copy the contents of the new array

```
private void reallocate () {  
    capacity *= 2;  
    theData = Arrays.copyOf(theData, capacity);  
}
```

reallocate Method (cont.)

```
private void reallocate () {  
    capacity *= 2;  
    theData = Arrays.copyOf(theData, capacity);  
}
```



The reason for doubling is to spread out the cost of copying; try to figure out why this is so!

DHArrayList as a Collection of Objects

- Earlier versions of Java did not support generics; all collections contained only `Object` elements
- To implement DHArrayList this way,
 - remove the parameter type `<E>` from the class heading,
 - replace each reference to data type `E` by `Object`
 - The underlying data array becomes `private Object[] theData;`

Vector Class

- The Java API `java.util` contains two very similar classes, `Vector` and `ArrayList`
- New applications normally use `ArrayList` rather than `Vector` as `ArrayList` is generally more efficient
- `Vector` class is *synchronized*, which means that multiple threads can access a `Vector` object without conflict

Interview Questions:

- Implement an algorithm to determine if a string has all unique characters.
 - What if you can not use additional data structures?

Interview Questions: Contd.

- Design an algorithm and write code to remove the duplicate characters in a string
 - without using any additional buffer
- Write a method to decide if two strings are anagrams or not.

- Questions..??